

Unifying Random Fourier Features and Leverage Scores for Kernel Matrix Approximation

Christopher Musco, Massachusetts Institute of Technology

ICML 2017:

“Random Fourier Features for Kernel Ridge Regression:
Approximation Bounds and Statistical Guarantees”

Joint work with:

Haim Avron (TAU)
Michael Kapralov (EPFL)
Cameron Musco (MIT)
Ameya Velingker (EPFL)
Amir Zandieh (EPFL)

Main idea:

Study **Fourier kernel approximation methods** from a **matrix sampling point of view**.

Main idea:

Study **Fourier kernel approximation methods** from a **matrix sampling point of view**.

Specific Contributions:

- Analyze Random Fourier Features method (Rahimi, Recht NIPS '07) using techniques based on **leverage scores**.

Main idea:

Study **Fourier kernel approximation methods** from a **matrix sampling point of view**.

Specific Contributions:

- Analyze Random Fourier Features method (Rahimi, Recht NIPS '07) using techniques based on **leverage scores**.
- Develop an **improved Random Fourier Features method** based on this analysis (better in theory and experiments).

Main idea:

Study **Fourier kernel approximation methods** from a **matrix sampling point of view**.

Specific Contributions:

- Analyze Random Fourier Features method (Rahimi, Recht NIPS '07) using techniques based on **leverage scores**.
- Develop an **improved Random Fourier Features method** based on this analysis (better in theory and experiments).

Lots of open questions and directions for future work.

Opportunities to combine techniques from randomized linear algebra and Fourier methods.

Quick refresher on kernel methods.

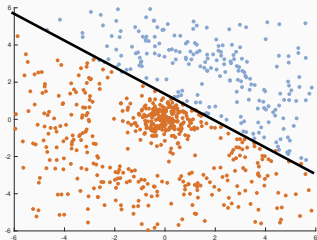
Kernel Methods in Machine Learning

Use **linear learning methods** (regression, support vector machines, PCA) to **learn nonlinear relationships**.

Kernel Methods in Machine Learning

Use **linear learning methods** (regression, support vector machines, PCA) to **learn nonlinear relationships**.

Linear Classifier

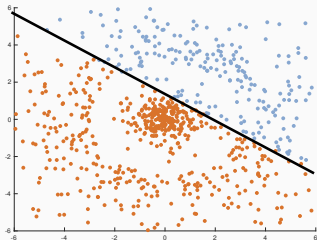


$$x_1 + 2x_2 \geq 6$$

Kernel Methods in Machine Learning

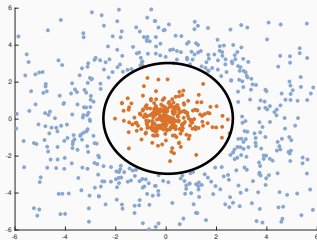
Use **linear learning methods** (regression, support vector machines, PCA) to **learn nonlinear relationships**.

Linear Classifier



$$x_1 + 2x_2 \geq 6$$

Kernel Classifier



$$x_1^2 + x_2^2 \geq 10$$

“Lift” data points to a higher dimensional feature space. E.g.

$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_d \end{bmatrix}$$

Kernel Methods in Machine Learning

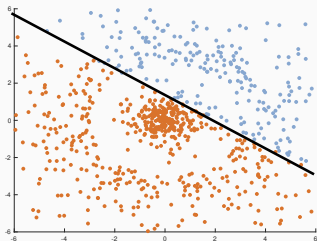
“Lift” data points to a higher dimensional feature space. E.g.

$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_d \end{bmatrix} \implies \phi(\mathbf{x}) = \begin{bmatrix} x_1 \\ \vdots \\ x_d \\ x_1 x_1 \\ x_1 x_2 \\ \vdots \\ x_d x_d \end{bmatrix}$$

Kernel Methods in Machine Learning

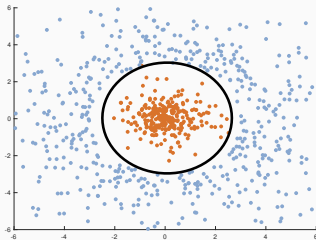
Use **linear learning methods** (regression, support vector machines, PCA) to **learn nonlinear relationships**.

Linear Classifier



$$x_1 + 2x_2 \geq 6$$

Kernel Classifier

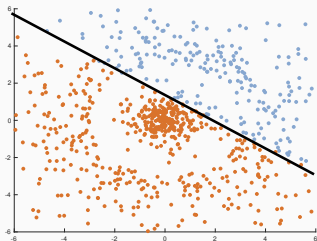


$$x_1^2 + x_2^2 \geq 10$$

Kernel Methods in Machine Learning

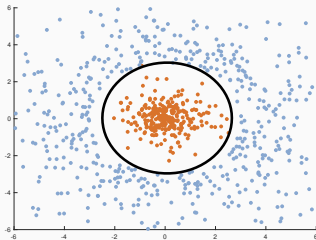
Use **linear learning methods** (regression, support vector machines, PCA) to **learn nonlinear relationships**.

Linear Classifier



$$x_1 + 2x_2 \geq 6$$

Kernel Classifier



$$x_1^2 + x_2^2 \geq 10$$

$$\phi(\mathbf{x})_3 + \phi(\mathbf{x})_4 \geq 10$$

The Cost of Kernel Methods

Main computational issue: Forming $\phi(\mathbf{x})$ is intractable even for moderately complex kernels.

E.g. degree q polynomials $\implies O(d^q)$ dimensional vectors.

The Cost of Kernel Methods

Main computational issue: Forming $\phi(\mathbf{x})$ is intractable even for moderately complex kernels.

E.g. degree q polynomials $\implies O(d^q)$ dimensional vectors.

Fix: For common linear learning methods, we only need the **kernel dot product** for each pair of data points $\mathbf{x}, \mathbf{y}$:

$$k(\mathbf{x}, \mathbf{y}) = \langle \phi(\mathbf{x}), \phi(\mathbf{y}) \rangle .$$

The Cost of Kernel Methods

Main computational issue: Forming $\phi(\mathbf{x})$ is intractable even for moderately complex kernels.

E.g. degree q polynomials $\implies O(d^q)$ dimensional vectors.

Fix: For common linear learning methods, we only need the **kernel dot product** for each pair of data points $\mathbf{x}, \mathbf{y}$:

$$k(\mathbf{x}, \mathbf{y}) = \langle \phi(\mathbf{x}), \phi(\mathbf{y}) \rangle .$$

Can often be computed much more quickly than $\phi(\mathbf{x}), \phi(\mathbf{y})$.

The “Kernel Trick”

Kernel dot product can often be computed **implicitly** without forming $\phi(\mathbf{x})$ and $\phi(\mathbf{y})$. For example:

$$\begin{aligned}(1 + \langle \mathbf{x}, \mathbf{y} \rangle)^2 &= (1 + x_1 y_1 + \dots + x_d y_d)^2 \\&= (1 + x_1 y_1 + x_1^2 y_1^2 + 2x_1 y_1 x_2 y_2 + \dots) \\&= \left\langle [1, x_1, x_1^2, \sqrt{2}x_1 x_2, \dots], [1, y_1, y_1^2, \sqrt{2}y_1 y_2, \dots] \right\rangle \\&= \langle \phi(\mathbf{x}), \phi(\mathbf{y}) \rangle\end{aligned}$$

The “Kernel Trick”

Kernel dot product can often be computed **implicitly** without forming $\phi(\mathbf{x})$ and $\phi(\mathbf{y})$. For example:

$$\begin{aligned}(1 + \langle \mathbf{x}, \mathbf{y} \rangle)^2 &= (1 + x_1 y_1 + \dots + x_d y_d)^2 \\&= (1 + x_1 y_1 + x_1^2 y_1^2 + 2x_1 y_1 x_2 y_2 + \dots) \\&= \left\langle [1, x_1, x_1^2, \sqrt{2}x_1 x_2, \dots], [1, y_1, y_1^2, \sqrt{2}y_1 y_2, \dots] \right\rangle \\&= \langle \phi(\mathbf{x}), \phi(\mathbf{y}) \rangle\end{aligned}$$

The corresponding feature map is sometimes less clear than our simple polynomial example.

The **kernel function** $k(\mathbf{x}, \mathbf{y}) = (1 + \mathbf{x}^T \mathbf{y})^2$ provides an alternative **similarity metric** to the standard dot product.

The “Kernel Trick”

Kernel learning pipeline for data points $\mathbf{x}_1, \dots, \mathbf{x}_n$:

1. Choose kernel function $k(\mathbf{x}_i, \mathbf{x}_j)$:

$$(1 + \mathbf{x}_i^T \mathbf{x}_j)^q, e^{-\|\mathbf{x}_i - \mathbf{x}_j\|^2}, e^{-\|\mathbf{x}_i - \mathbf{x}_j\|_1}, \text{ etc.}$$

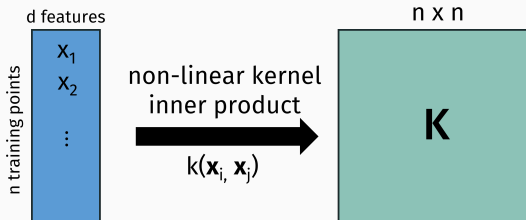
2. Form $n \times n$ **kernel matrix** $\mathbf{K}$ with:

$$\mathbf{K}_{i,j} = k(\mathbf{x}_i, \mathbf{x}_j)$$

3. Compute model using $\mathbf{K}$: compute $(\mathbf{K} + \lambda \mathbf{I})^{-1} \mathbf{b}$ for kernel regression, eigendecomposition of $\mathbf{K}$ for kernel PCA, etc.

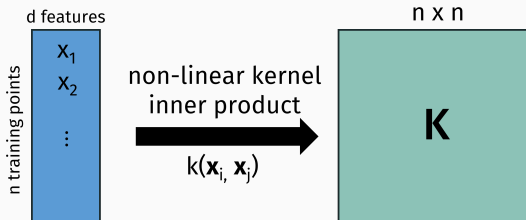
Algorithmic Challenge

Even if we avoid explicit feature expansion kernel methods are really slow... quadratic dependence on the number of data points.



Algorithmic Challenge

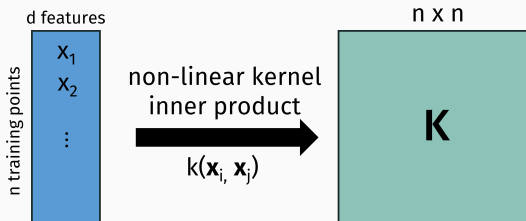
Even if we avoid explicit feature expansion kernel methods are really slow... quadratic dependence on the number of data points.



- $n = 100,000 \Rightarrow 10$ billion entries $\Rightarrow 80$ GB to store $\mathbf{K}$.

Algorithmic Challenge

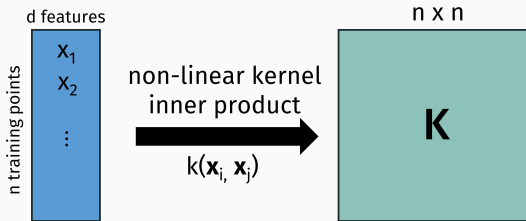
Even if we avoid explicit feature expansion kernel methods are really slow... quadratic dependence on the number of data points.



- $n = 100,000 \Rightarrow 10$ billion entries $\Rightarrow 80$ GB to store $\mathbf{K}$.
- Just writing down $\mathbf{K}$ requires $\Omega(n^2)$ time.

Algorithmic Challenge

Even if we avoid explicit feature expansion kernel methods are really slow... quadratic dependence on the number of data points.

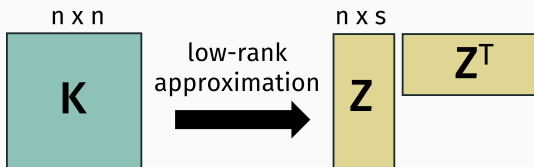


- $n = 100,000 \Rightarrow 10$ billion entries $\Rightarrow 80$ GB to store $\mathbf{K}$.
- Just writing down $\mathbf{K}$ requires $\Omega(n^2)$ time.
- Other operations require even more. A single iteration for a linear system solver takes $\Omega(n^2)$ time.

**New algorithmic ideas are needed to scale kernel methods.
Even for moderately large datasets.**

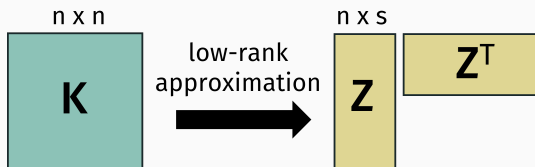
Standard Approach: Low-rank Approximation

Find approximation $\tilde{\mathbf{K}} = \mathbf{Z}\mathbf{Z}^T$ for $\mathbf{K}$ (which is symmetric and positive semidefinite):



Standard Approach: Low-rank Approximation

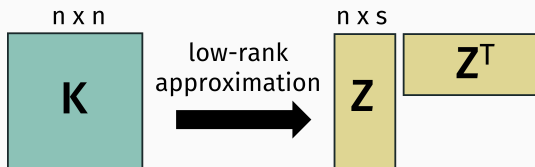
Find approximation $\tilde{\mathbf{K}} = \mathbf{Z}\mathbf{Z}^T$ for $\mathbf{K}$ (which is symmetric and positive semidefinite):



We can typically set $s \ll n$.

Standard Approach: Low-rank Approximation

Find approximation $\tilde{\mathbf{K}} = \mathbf{Z}\mathbf{Z}^T$ for $\mathbf{K}$ (which is symmetric and positive semidefinite):

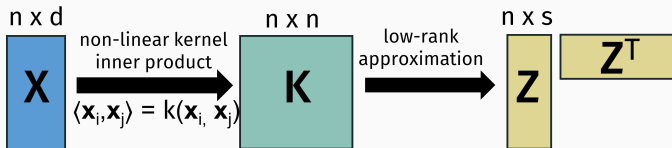


We can typically set $s \ll n$.

- $\mathbf{Z}$ takes $O(ns)$ space to store.
- Orthogonalization, eigendecomposition, and inversion of $\mathbf{Z}\mathbf{Z}^T$ all take just $O(ns^2)$ time.
- $\mathbf{Z}\mathbf{Z}^T \mathbf{x}$ can be computed in $O(ns)$ time.

Implicit Low-rank Approximation

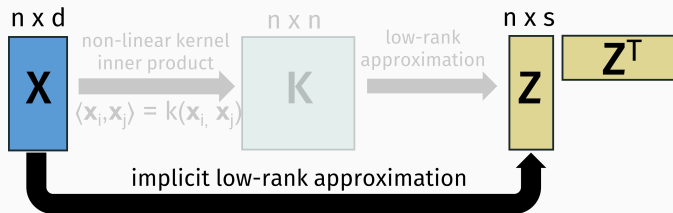
All standard low-rank approximation algorithms take $\Omega(n^2)$ time for kernel matrices: we at least need to compute $\mathbf{K}$.



Goal: Find methods that avoid explicit access to the kernel matrix.

Implicit Low-rank Approximation

All standard low-rank approximation algorithms take $\Omega(n^2)$ time for kernel matrices: we at least need to compute $\mathbf{K}$.



Goal: Find methods that avoid explicit access to the kernel matrix.

Suppose $k(\mathbf{x}, \mathbf{y})$ is **shift invariant**.

Suppose $k(\mathbf{x}, \mathbf{y})$ is **shift invariant**. I.e.

$$k(\mathbf{x}, \mathbf{y}) = k(\mathbf{\Delta}) \quad \text{where} \quad \mathbf{\Delta} = \mathbf{x} - \mathbf{y}.$$

Suppose $k(\mathbf{x}, \mathbf{y})$ is **shift invariant**. I.e.

$$k(\mathbf{x}, \mathbf{y}) = k(\mathbf{\Delta}) \quad \text{where} \quad \mathbf{\Delta} = \mathbf{x} - \mathbf{y}.$$

Many common kernels are:

Suppose $k(\mathbf{x}, \mathbf{y})$ is **shift invariant**. I.e.

$$k(\mathbf{x}, \mathbf{y}) = k(\mathbf{\Delta}) \quad \text{where} \quad \mathbf{\Delta} = \mathbf{x} - \mathbf{y}.$$

Many common kernels are:

- Gaussian (RBF) kernel: $k(\mathbf{x}, \mathbf{y}) = e^{-\|\mathbf{x} - \mathbf{y}\|^2}$

Suppose $k(\mathbf{x}, \mathbf{y})$ is **shift invariant**. I.e.

$$k(\mathbf{x}, \mathbf{y}) = k(\mathbf{\Delta}) \quad \text{where} \quad \mathbf{\Delta} = \mathbf{x} - \mathbf{y}.$$

Many common kernels are:

- Gaussian (RBF) kernel: $k(\mathbf{x}, \mathbf{y}) = e^{-\|\mathbf{x}-\mathbf{y}\|^2}$
- Laplace kernel: $k(\mathbf{x}, \mathbf{y}) = e^{-\|\mathbf{x}-\mathbf{y}\|_1}$

Suppose $k(\mathbf{x}, \mathbf{y})$ is **shift invariant**. I.e.

$$k(\mathbf{x}, \mathbf{y}) = k(\mathbf{\Delta}) \quad \text{where} \quad \mathbf{\Delta} = \mathbf{x} - \mathbf{y}.$$

Many common kernels are:

- Gaussian (RBF) kernel: $k(\mathbf{x}, \mathbf{y}) = e^{-\|\mathbf{x}-\mathbf{y}\|^2}$
- Laplace kernel: $k(\mathbf{x}, \mathbf{y}) = e^{-\|\mathbf{x}-\mathbf{y}\|_1}$
- Matern kernel, Cauchy kernel, rational quadratic kernel, etc.

Suppose $k(\mathbf{x}, \mathbf{y})$ is **shift invariant**. I.e.

$$k(\mathbf{x}, \mathbf{y}) = k(\mathbf{\Delta}) \quad \text{where} \quad \mathbf{\Delta} = \mathbf{x} - \mathbf{y}.$$

Many common kernels are:

- Gaussian (RBF) kernel: $k(\mathbf{x}, \mathbf{y}) = e^{-\|\mathbf{x}-\mathbf{y}\|^2}$
- Laplace kernel: $k(\mathbf{x}, \mathbf{y}) = e^{-\|\mathbf{x}-\mathbf{y}\|_1}$
- Matern kernel, Cauchy kernel, rational quadratic kernel, etc.

Random Fourier Features (RFF) Algorithm

Write $k(\Delta)$ using its inverse Fourier transform,

$$p(\eta) = \mathcal{F}^{-1}(k) \qquad k(\Delta) = \int_{\eta \in \mathbb{R}^d} p(\eta) e^{-i\pi \eta^T \Delta} d\eta$$

Random Fourier Features (RFF) Algorithm

Write $k(\Delta)$ using its inverse Fourier transform,

$$p(\eta) = \mathcal{F}^{-1}(k) \quad k(\Delta) = \int_{\eta \in \mathbb{R}^d} p(\eta) e^{-i\pi \eta^T \Delta} d\eta$$

Approximate with finite sum for any Δ :

$$k(\Delta) \approx \frac{1}{s} \sum_{j=1}^s c_j e^{-i\pi \eta_j^T \Delta}$$

Random Fourier Features (RFF) Algorithm

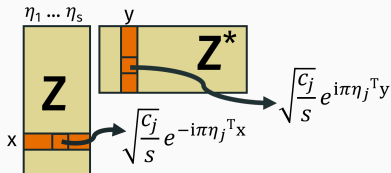
Write $k(\Delta)$ using its inverse Fourier transform,

$$p(\eta) = \mathcal{F}^{-1}(k) \quad k(\Delta) = \int_{\eta \in \mathbb{R}^d} p(\eta) e^{-i\pi \eta^T \Delta} d\eta$$

Approximate with finite sum for any Δ :

$$k(\Delta) \approx \frac{1}{s} \sum_{j=1}^s c_j e^{-i\pi \eta_j^T \Delta}$$

Immediately gives low-rank approximation:



Random Fourier Features (RFF) Algorithm

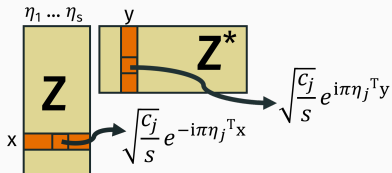
Write $k(\Delta)$ using its inverse Fourier transform,

$$p(\eta) = \mathcal{F}^{-1}(k) \quad k(\Delta) = \int_{\eta \in \mathbb{R}^d} p(\eta) e^{-i\pi \eta^T \Delta} d\eta$$

Approximate with finite sum for any Δ :

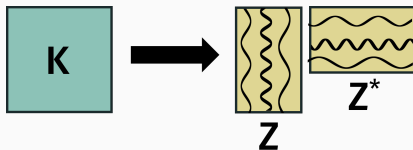
$$k(\Delta) \approx \frac{1}{s} \sum_{j=1}^s c_j e^{-i\pi \eta_j^T \Delta}$$

Immediately gives low-rank approximation:



$$\begin{aligned} \mathbf{Z}(\mathbf{x})\mathbf{Z}(\mathbf{y})^* &= \sum_j \frac{c_j}{s} e^{-i\pi \eta_j^T (\mathbf{x} - \mathbf{y})} \\ &\approx k(\mathbf{x} - \mathbf{y}) \end{aligned}$$

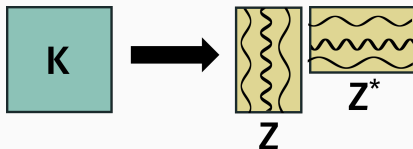
Random Fourier Features (RFF) Algorithm



How do we choose the frequencies in the finite sum?

$$\int_{\boldsymbol{\eta} \in \mathbb{R}^d} p(\boldsymbol{\eta}) e^{-i\pi \boldsymbol{\eta}^T \boldsymbol{\Delta}} d\boldsymbol{\eta} \approx \frac{1}{s} \sum_{j=1}^s c_j e^{-i\pi \boldsymbol{\eta}_j^T \boldsymbol{\Delta}}$$

Random Fourier Features (RFF) Algorithm



How do we choose the frequencies in the finite sum?

$$\int_{\boldsymbol{\eta} \in \mathbb{R}^d} p(\boldsymbol{\eta}) e^{-i\pi \boldsymbol{\eta}^T \boldsymbol{\Delta}} d\boldsymbol{\eta} \approx \frac{1}{s} \sum_{j=1}^s c_j e^{-i\pi \boldsymbol{\eta}_j^T \boldsymbol{\Delta}}$$

Bochner's Theorem: For shift-invariant, positive semidefinite kernel functions, $p(\boldsymbol{\eta}) \geq 0$ for all $\boldsymbol{\eta}$.

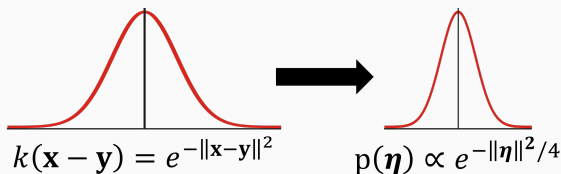
Random Fourier Features (RFF) Algorithm

Bochner's Theorem: For shift-invariant, positive semidefinite kernel functions, $p(\boldsymbol{\eta}) \geq 0$ for all $\boldsymbol{\eta}$.

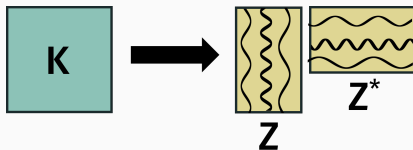
Random Fourier Features (RFF) Algorithm

Bochner's Theorem: For shift-invariant, positive semidefinite kernel functions, $p(\boldsymbol{\eta}) \geq 0$ for all $\boldsymbol{\eta}$.

Example (Gaussian kernel):

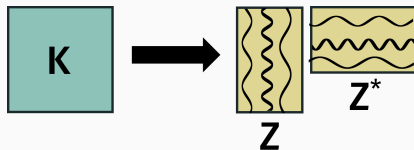


Random Fourier Features (RFF) Algorithm



Sample frequencies from distribution p . Set $c_j = 1$ for all j .

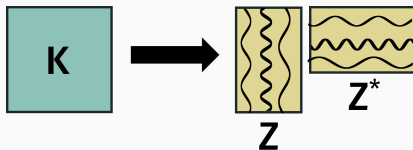
Random Fourier Features (RFF) Algorithm



Sample frequencies from distribution p . Set $c_j = 1$ for all j .

$$\mathbb{E} \left[\frac{1}{s} \sum_{j=1}^s e^{-i\pi \eta_j^T \Delta} \right] = \mathbb{E} \left[e^{-i\pi \eta^T \Delta} \right]$$

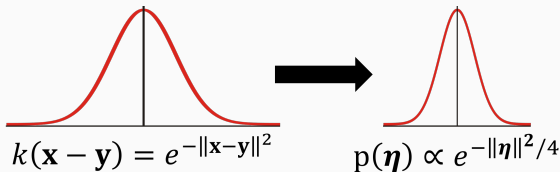
Random Fourier Features (RFF) Algorithm



Sample frequencies from distribution p . Set $c_j = 1$ for all j .

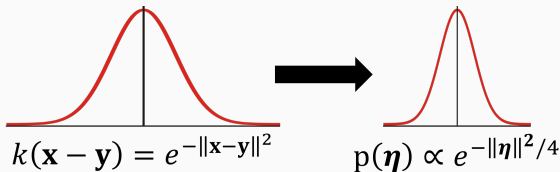
$$\mathbb{E} \left[\frac{1}{s} \sum_{j=1}^s e^{-i\pi \eta_j^T \Delta} \right] = \mathbb{E} \left[e^{-i\pi \eta^T \Delta} \right] = \int_{\eta \in \mathbb{R}^d} p(\eta) e^{-i\pi \eta^T \Delta} d\eta$$

Random Fourier Features (RFF) Algorithm



- Sampling frequencies by $p(\boldsymbol{\eta})$ is correct in expectation.

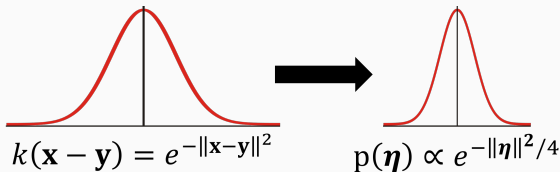
Random Fourier Features (RFF) Algorithm



The diagram illustrates the transformation in the Random Fourier Features (RFF) algorithm. On the left, a red Gaussian curve represents the kernel $k(\mathbf{x} - \mathbf{y}) = e^{-\|\mathbf{x} - \mathbf{y}\|^2}$. A thick black arrow points to the right, where another red Gaussian curve represents the sampling distribution $p(\boldsymbol{\eta}) \propto e^{-\|\boldsymbol{\eta}\|^2/4}$. Both curves are centered on a horizontal axis with a vertical line indicating the peak.

- Sampling frequencies by $p(\boldsymbol{\eta})$ is correct in expectation.
- Final sum $\frac{1}{s} \sum_{j=1}^s e^{-i\pi \boldsymbol{\eta}_j^T \boldsymbol{\Delta}} \approx k(\boldsymbol{\Delta})$ only includes terms with magnitude 1. Real component ≤ 1 . Imaginary component ≤ 1 .

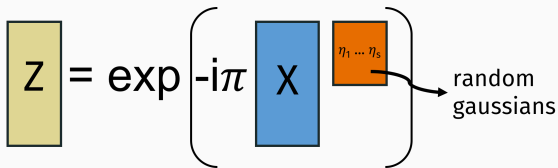
Random Fourier Features (RFF) Algorithm



- Sampling frequencies by $p(\boldsymbol{\eta})$ is correct in expectation.
- Final sum $\frac{1}{s} \sum_{j=1}^s e^{-i\pi \boldsymbol{\eta}_j^T \boldsymbol{\Delta}} \approx k(\boldsymbol{\Delta})$ only includes terms with magnitude 1. Real component ≤ 1 . Imaginary component ≤ 1 .
- By Chernoff bound, if we take $O\left(\frac{\log n}{\epsilon^2}\right)$ samples, we approximate every entry in $\mathbf{K}$ to error $\pm\epsilon$.

Random Fourier Features (RFF) Algorithm

Super simple algorithm. For Gaussian kernel $k(\mathbf{x} - \mathbf{y}) = e^{-\|\mathbf{x} - \mathbf{y}\|_2^2}$:



The diagram illustrates the RFF algorithm equation: $Z = \exp\left(-i\pi X G\right)$. The variable Z is represented by a yellow rectangle. The variable X is represented by a blue rectangle. The variable G is represented by an orange rectangle containing the text $\eta_1 \dots \eta_s$. An arrow points from the orange rectangle to the text "random gaussians". The entire expression is enclosed in large parentheses.

```
G = randn(d, s);
```

```
Z = exp(-sqrt(-1)*pi*X*G)/sqrt(s);
```


Our Starting Point

Want a better understanding of (and to ideally improve) the approximation guarantee:

$$\begin{matrix} \boxed{Z} \end{matrix} \begin{matrix} \boxed{Z^*} \end{matrix} = \begin{matrix} \boxed{K} \end{matrix} + \begin{matrix} \boxed{\begin{matrix} \pm\epsilon & \pm\epsilon & \pm\epsilon & \pm\epsilon \\ \pm\epsilon & \pm\epsilon & \pm\epsilon & \pm\epsilon \\ \pm\epsilon & \pm\epsilon & \pm\epsilon & \pm\epsilon \\ \pm\epsilon & \pm\epsilon & \pm\epsilon & \pm\epsilon \end{matrix}} \end{matrix} \begin{matrix} \boxed{E} \end{matrix}$$

Our Starting Point

Want a better understanding of (and to ideally improve) the approximation guarantee:

$$\begin{array}{|c|} \hline \mathbf{Z} \\ \hline \end{array} \begin{array}{|c|} \hline \mathbf{Z}^* \\ \hline \end{array} = \begin{array}{|c|} \hline \mathbf{K} \\ \hline \end{array} + \begin{array}{|c|} \hline \begin{array}{cccc} \pm\epsilon & \pm\epsilon & \pm\epsilon & \pm\epsilon \\ \pm\epsilon & \pm\epsilon & \pm\epsilon & \pm\epsilon \\ \pm\epsilon & \pm\epsilon & \pm\epsilon & \pm\epsilon \\ \pm\epsilon & \pm\epsilon & \pm\epsilon & \pm\epsilon \end{array} \\ \hline \end{array}$$

- Faster but less accurate than e.g. a Nyström approximation with the same number of samples.

Our Starting Point

Want a better understanding of (and to ideally improve) the approximation guarantee:

$$\mathbf{Z} \mathbf{Z}^* = \mathbf{K} + \mathbf{E}$$

- Faster but less accurate than e.g. a Nyström approximation with the same number of samples.
- Does not give good bounds on $\|\mathbf{Z}\mathbf{Z}^* - \mathbf{K}\|$ unless we take $\Omega(n^2)$ samples.

Our Starting Point

Want a better understanding of (and to ideally improve) the approximation guarantee:

$$\mathbf{Z} \mathbf{Z}^* = \mathbf{K} + \mathbf{E}$$

- Faster but less accurate than e.g. a Nyström approximation with the same number of samples.
- Does not give good bounds on $\|\mathbf{Z}\mathbf{Z}^* - \mathbf{K}\|$ unless we take $\Omega(n^2)$ samples.
- No clear implications for downstream learning tasks – e.g. does $(\mathbf{Z}\mathbf{Z}^* + \lambda\mathbf{I})^{-1}$ approximate $(\mathbf{K} + \lambda\mathbf{I})^{-1}$.

Our Starting Point

In short, we want matrix-like error bounds, not entrywise bounds.

Our Starting Point

In short, we want matrix-like error bounds, not entrywise bounds.

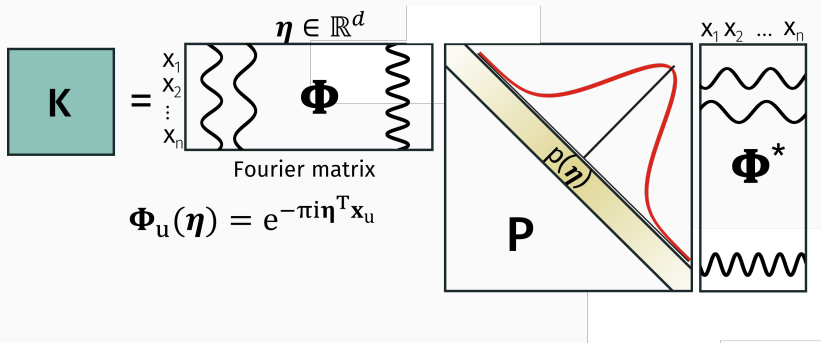
Key Idea: Analyze the random Fourier features algorithm as a **matrix sampling process**.

High level approach

$$k(\mathbf{x}_u - \mathbf{x}_v) = \int_{\boldsymbol{\eta} \in \mathbb{R}^d} p(\boldsymbol{\eta}) e^{-i\pi \boldsymbol{\eta}^T (\mathbf{x}_u - \mathbf{x}_v)} d\boldsymbol{\eta}$$

High level approach

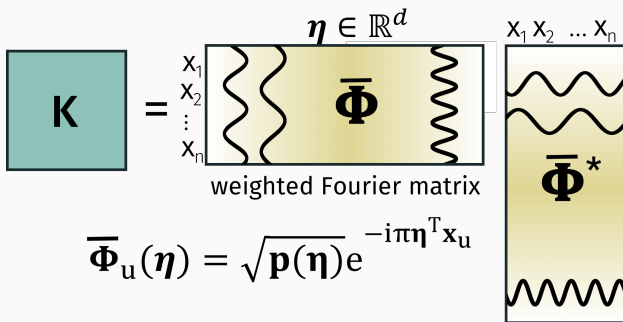
$$k(\mathbf{x}_u - \mathbf{x}_v) = \int_{\boldsymbol{\eta} \in \mathbb{R}^d} \rho(\boldsymbol{\eta}) e^{-i\pi \boldsymbol{\eta}^T (\mathbf{x}_u - \mathbf{x}_v)} d\boldsymbol{\eta}$$



Kernel Fourier Transform

High level approach

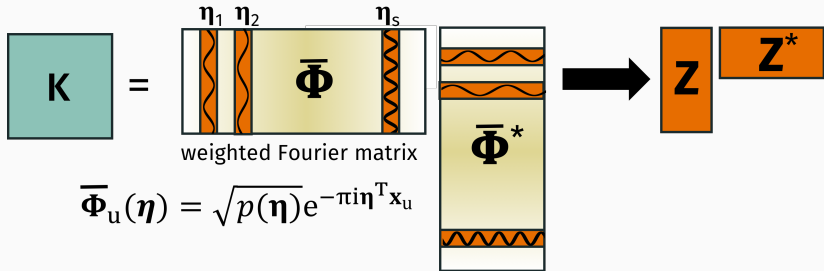
$$k(\mathbf{x}_u - \mathbf{x}_v) = \int_{\boldsymbol{\eta} \in \mathbb{R}^d} \rho(\boldsymbol{\eta}) e^{-i\pi \boldsymbol{\eta}^T (\mathbf{x}_u - \mathbf{x}_v)} d\boldsymbol{\eta}$$



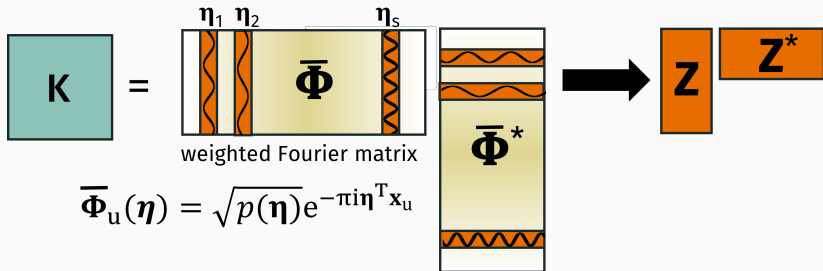
$$\bar{\boldsymbol{\Phi}}_u(\boldsymbol{\eta}) = \sqrt{\mathbf{p}(\boldsymbol{\eta})} e^{-i\pi \boldsymbol{\eta}^T \mathbf{x}_u}$$

Kernel Fourier Transform

Standard RFF = Column Norm Sampling

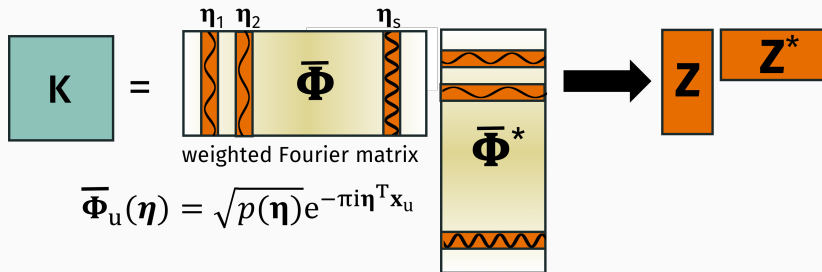


Standard RFF = Column Norm Sampling



Standard RFF selects column $\bar{\Phi}(\eta)$ with probability $\propto p(\eta)$.

Standard RFF = Column Norm Sampling



Standard RFF selects column $\bar{\Phi}(\eta)$ with probability $\propto p(\eta)$. I.e. with probability proportional to the column's **squared norm**.

Leverage Score Sampling

For matrix approximation, norm based sampling probabilities are known to be suboptimal!

Leverage Score Sampling

For matrix approximation, norm based sampling probabilities are known to be suboptimal!

We can obtain better theoretical bounds and empirical performance if we sample by column **leverage scores.**

(see work on spectral graph sparsification, matrix concentration bounds, randomized linear algebra, etc.)

Comparison of Approximation Guarantees

Column Norm Sampling: $s = O(\log n/\epsilon^2)$ samples ensure that $(\mathbf{Z}\mathbf{Z}^*)_{i,j} = \mathbf{K}_{i,j} \pm \epsilon$ for all i,j with high probability.

Comparison of Approximation Guarantees

Column Norm Sampling: $s = O(\log n/\epsilon^2)$ samples ensure that $(\mathbf{Z}\mathbf{Z}^*)_{i,j} = \mathbf{K}_{i,j} \pm \epsilon$ for all i, j with high probability.

Ridge Leverage Score Sampling:

$$(1 - \epsilon)(\mathbf{Z}\mathbf{Z}^* + \lambda\mathbf{I}) \preceq \mathbf{K} + \lambda\mathbf{I} \preceq (1 + \epsilon)(\mathbf{Z}\mathbf{Z}^* + \lambda\mathbf{I}).$$

with $\tilde{O}(s_\lambda/\epsilon^2)$ samples where $s_\lambda = \text{tr}(\mathbf{K}(\mathbf{K} + \lambda\mathbf{I})^{-1})$ is the **statistical dimension**.

Comparison of Approximation Guarantees

Column Norm Sampling: $s = O(\log n/\epsilon^2)$ samples ensure that $(\mathbf{Z}\mathbf{Z}^*)_{i,j} = \mathbf{K}_{i,j} \pm \epsilon$ for all i, j with high probability.

Ridge Leverage Score Sampling:

$$(1 - \epsilon)(\mathbf{Z}\mathbf{Z}^* + \lambda\mathbf{I}) \preceq \mathbf{K} + \lambda\mathbf{I} \preceq (1 + \epsilon)(\mathbf{Z}\mathbf{Z}^* + \lambda\mathbf{I}).$$

with $\tilde{O}(s_\lambda/\epsilon^2)$ samples where $s_\lambda = \text{tr}(\mathbf{K}(\mathbf{K} + \lambda\mathbf{I})^{-1})$ is the **statistical dimension**.

- Since $A \preceq B \Rightarrow B^{-1} \preceq A^{-1}$, $(\mathbf{Z}\mathbf{Z}^* + \lambda\mathbf{I})^{-1} \approx (\mathbf{K} + \lambda\mathbf{I})^{-1}$

Comparison of Approximation Guarantees

Column Norm Sampling: $s = O(\log n/\epsilon^2)$ samples ensure that $(\mathbf{Z}\mathbf{Z}^*)_{i,j} = \mathbf{K}_{i,j} \pm \epsilon$ for all i, j with high probability.

Ridge Leverage Score Sampling:

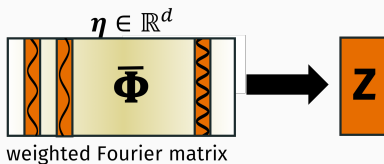
$$(1 - \epsilon)(\mathbf{Z}\mathbf{Z}^* + \lambda\mathbf{I}) \preceq \mathbf{K} + \lambda\mathbf{I} \preceq (1 + \epsilon)(\mathbf{Z}\mathbf{Z}^* + \lambda\mathbf{I}).$$

with $\tilde{O}(s_\lambda/\epsilon^2)$ samples where $s_\lambda = \text{tr}(\mathbf{K}(\mathbf{K} + \lambda\mathbf{I})^{-1})$ is the **statistical dimension**.

- Since $A \preceq B \Rightarrow B^{-1} \preceq A^{-1}$, $(\mathbf{Z}\mathbf{Z}^* + \lambda\mathbf{I})^{-1} \approx (\mathbf{K} + \lambda\mathbf{I})^{-1}$
- Spectral approximation also gives approximation bounds for kernel PCA, k-means, CCA, etc. (Cohen, Musco, M '16, '17)

Computing Ridge Leverage Scores

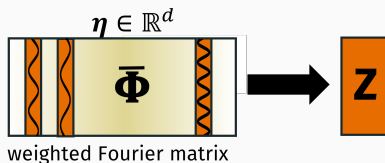
What are the ridge leverage scores?



$$\tau_{\lambda}(\eta) = \bar{\Phi}(\eta)^* (\mathbf{K} + \lambda \mathbf{I})^{-1} \bar{\Phi}(\eta).$$

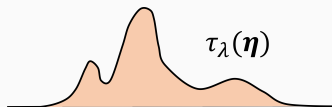
Computing Ridge Leverage Scores

What are the ridge leverage scores?



$$\tau_{\lambda}(\eta) = \bar{\Phi}(\eta)^* (\mathbf{K} + \lambda \mathbf{I})^{-1} \bar{\Phi}(\eta).$$

Expensive to invert $\mathbf{K} + \lambda \mathbf{I}$. Even if you could, not at all clear how to efficiently sample from the leverage score distribution.

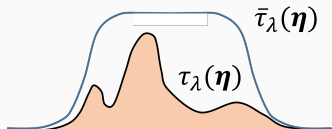


Bounding the Fourier Leverage Scores

Main goal: Get a handle on the Fourier ridge leverage scores for common kernels by upper bounding them with simple distributions.

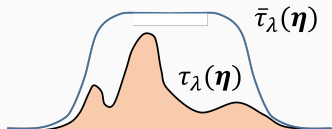
Bounding the Fourier Leverage Scores

Main goal: Get a handle on the Fourier ridge leverage scores for common kernels by upper bounding them with simple distributions.



Bounding the Fourier Leverage Scores

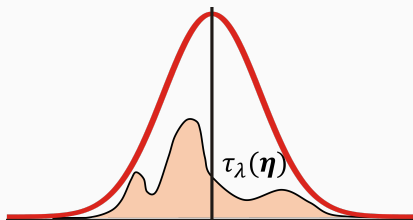
Main goal: Get a handle on the Fourier ridge leverage scores for common kernels by upper bounding them with simple distributions.



More closely match leverage score sampling to improve random Fourier features.

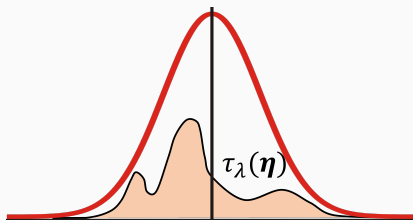
Bounding the Fourier Leverage Scores

First observation: Scaled by $\frac{n}{\lambda}$, the standard Rahimi Recht sampling distribution upper bounds the λ ridge leverage scores.



Bounding the Fourier Leverage Scores

First observation: Scaled by $\frac{n}{\lambda}$, the standard Rahimi Recht sampling distribution upper bounds the λ ridge leverage scores.



Basic Result: Sampling $O(\frac{n}{\lambda} \cdot \frac{1}{\epsilon^2})$ frequencies with RFF gives spectral guarantees.

Alternative Characterization of Leverage Scores

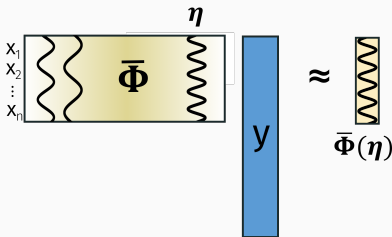
Ridge leverage score solves:

$$\tau_\lambda(\boldsymbol{\eta}) = \min_{\mathbf{y}} \left[\frac{1}{\lambda} \|\bar{\boldsymbol{\Phi}}\mathbf{y} - \bar{\boldsymbol{\Phi}}(\boldsymbol{\eta})\|_2^2 + \|\mathbf{y}\|_2^2 \right].$$

Alternative Characterization of Leverage Scores

Ridge leverage score solves:

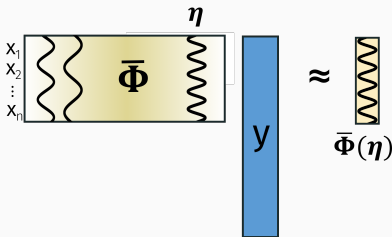
$$\tau_{\lambda}(\boldsymbol{\eta}) = \min_{\mathbf{y}} \left[\frac{1}{\lambda} \|\bar{\Phi} \mathbf{y} - \bar{\Phi}(\boldsymbol{\eta})\|_2^2 + \|\mathbf{y}\|_2^2 \right].$$



Alternative Characterization of Leverage Scores

Ridge leverage score solves:

$$\tau_{\lambda}(\boldsymbol{\eta}) = \min_{\mathbf{y}} \left[\frac{1}{\lambda} \|\bar{\Phi} \mathbf{y} - \bar{\Phi}(\boldsymbol{\eta})\|_2^2 + \|\mathbf{y}\|_2^2 \right].$$

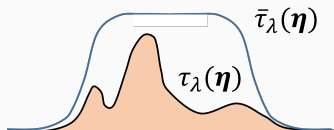


Intuition: $\mathbf{y}$ reconstructs frequency $\boldsymbol{\eta}$ from other frequencies. The easier $\boldsymbol{\eta}$ is to reconstruct, the less important it is to sample.

Bounding the Fourier Leverage Scores

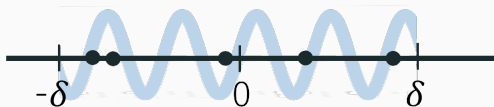
Approach: Obtain closed form upper bound on leverage scores by exhibiting simple candidate vector $\tilde{\mathbf{y}}$ and noting that:

$$\tau_{\lambda}(\boldsymbol{\eta}) \leq \frac{1}{\lambda} \|\bar{\boldsymbol{\Phi}}\tilde{\mathbf{y}} - \bar{\boldsymbol{\Phi}}(\boldsymbol{\eta})\|_2^2 + \|\tilde{\mathbf{y}}\|_2^2.$$



Simplifying Assumption

For rest of talk: Assume Gaussian kernel, n points in 1-dimension, bounded between $[-\delta, \delta]$.



First attempt

Need to bound: $\frac{1}{\lambda} \|\bar{\Phi} \tilde{\mathbf{y}} - \bar{\Phi}(\boldsymbol{\eta})\|_2^2 + \|\tilde{\mathbf{y}}\|_2^2.$

First attempt

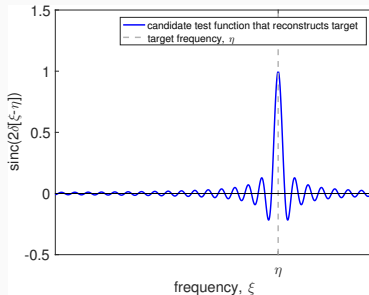
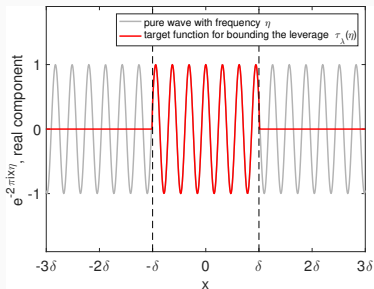
Need to bound: $\frac{1}{\lambda} \|\bar{\Phi} \tilde{\mathbf{y}} - \bar{\Phi}(\boldsymbol{\eta})\|_2^2 + \|\tilde{\mathbf{y}}\|_2^2$.

Set $\tilde{\mathbf{y}}(f)$ to be a **shifted sinc function**, weighted by $1/\sqrt{p(f)}$.

First attempt

Need to bound: $\frac{1}{\lambda} \|\bar{\Phi}\tilde{\mathbf{y}} - \bar{\Phi}(\eta)\|_2^2 + \|\tilde{\mathbf{y}}\|_2^2$.

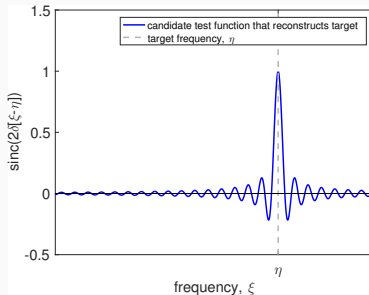
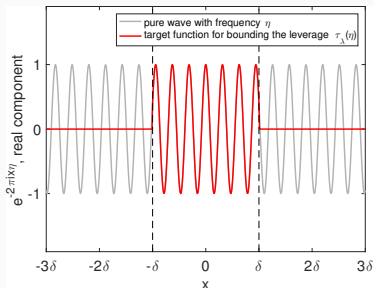
Set $\tilde{\mathbf{y}}(f)$ to be a **shifted sinc function**, weighted by $1/\sqrt{p(f)}$.



First attempt

Need to bound: $\frac{1}{\lambda} \|\bar{\Phi}\tilde{\mathbf{y}} - \bar{\Phi}(\eta)\|_2^2 + \|\tilde{\mathbf{y}}\|_2^2$.

Set $\tilde{\mathbf{y}}(f)$ to be a **shifted sinc function**, weighted by $1/\sqrt{p(f)}$.

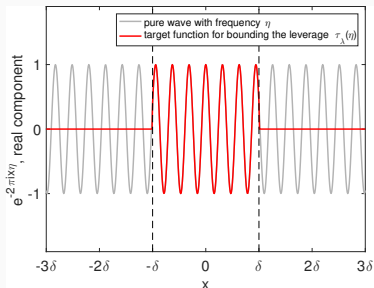


$$\frac{1}{\lambda} \|\bar{\Phi}\tilde{\mathbf{y}} - \bar{\Phi}(\eta)\|_2^2 = 0!$$

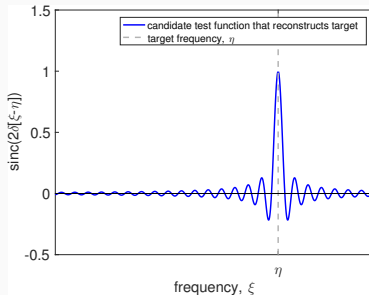
First attempt

Need to bound: $\frac{1}{\lambda} \|\bar{\Phi} \tilde{\mathbf{y}} - \bar{\Phi}(\eta)\|_2^2 + \|\tilde{\mathbf{y}}\|_2^2$.

Set $\tilde{\mathbf{y}}(f)$ to be a **shifted sinc function**, weighted by $1/\sqrt{p(f)}$.



$$\frac{1}{\lambda} \|\bar{\Phi} \tilde{\mathbf{y}} - \bar{\Phi}(\eta)\|_2^2 = 0!$$

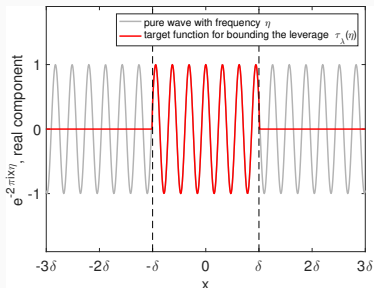


$$\|\tilde{\mathbf{y}}\|_2^2 = O(\delta).$$

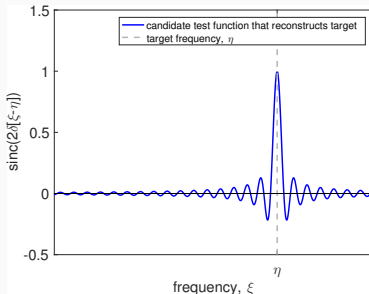
First attempt

Need to bound: $\frac{1}{\lambda} \|\bar{\Phi} \tilde{\mathbf{y}} - \bar{\Phi}(\eta)\|_2^2 + \|\tilde{\mathbf{y}}\|_2^2$.

Set $\tilde{\mathbf{y}}(f)$ to be a **shifted sinc function**, weighted by $1/\sqrt{p(f)}$.



$$\frac{1}{\lambda} \|\bar{\Phi} \tilde{\mathbf{y}} - \bar{\Phi}(\eta)\|_2^2 = 0!$$

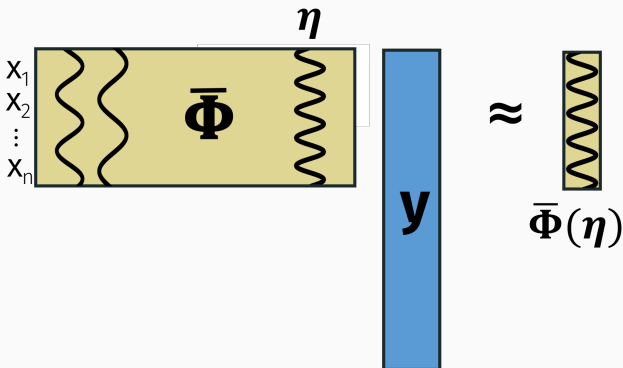


$$\|\tilde{\mathbf{y}}\|_2^2 = O(\delta).$$

$$\|\sqrt{p(f)} \tilde{\mathbf{y}}\|_2^2 = O(\delta), \quad \|\tilde{\mathbf{y}}\|_2^2 \text{ is much larger!}$$

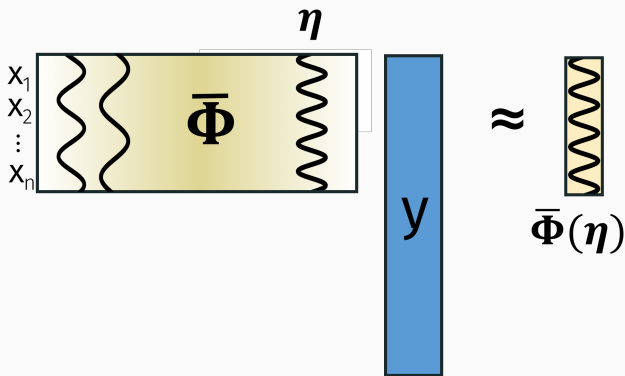
First attempt

Optimization problem is over **weighted** Fourier matrix.



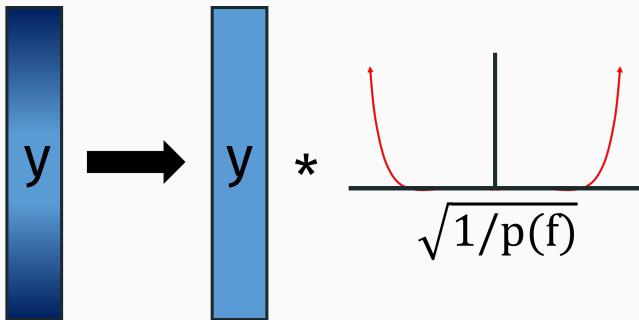
First attempt

Optimization problem is over **weighted** Fourier matrix.



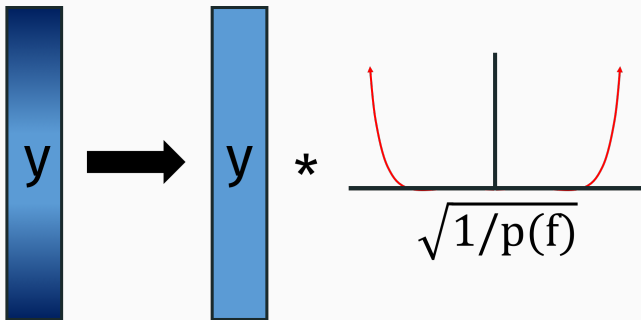
First attempt

Optimization problem is over **weighted** Fourier matrix.



First attempt

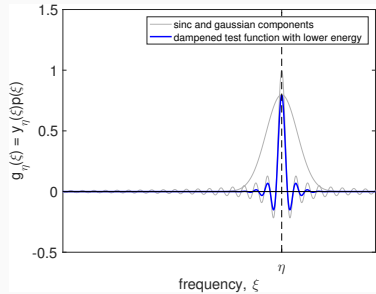
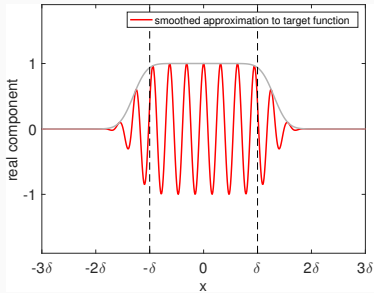
Optimization problem is over **weighted** Fourier matrix.



Sinc falls off as $O(1/f)$, but $1/\sqrt{p(f)}$ grows as $e^{O(f^2)}$, so $\|y\|_2^2$ explodes.

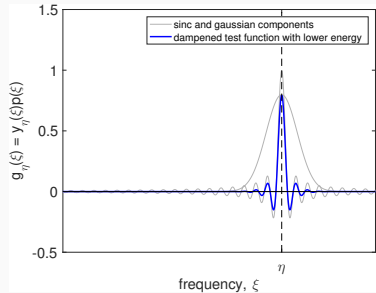
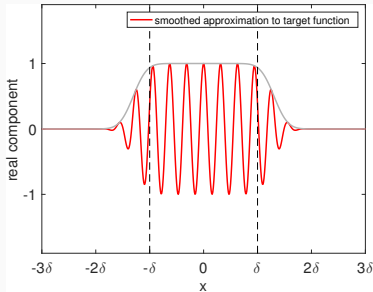
Final Test Function

Solution: Dampen sinc with a narrow Gaussian.



Final Test Function

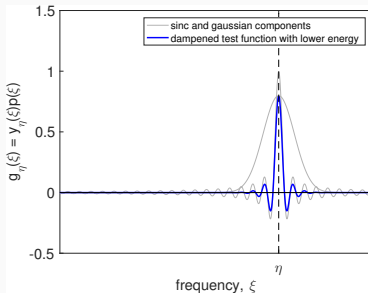
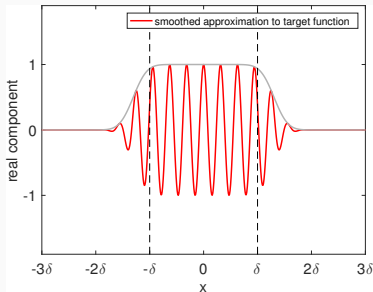
Solution: Dampen sinc with a narrow Gaussian.



$\frac{1}{\lambda} \|\bar{\Phi} \tilde{\mathbf{y}} - \bar{\Phi}(\eta)\|_2^2$ remains very small, $\|\mathbf{y}\|_2^2 \approx O(\delta)$.

Final Test Function

Solution: Dampen sinc with a narrow Gaussian.



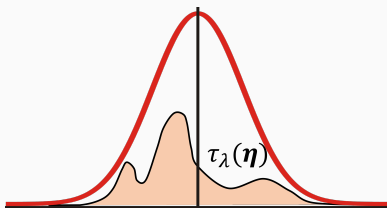
$$\frac{1}{\lambda} \|\bar{\Phi} \tilde{\mathbf{y}} - \bar{\Phi}(\eta)\|_2^2 \text{ remains very small, } \|\mathbf{y}\|_2^2 \approx O(\delta).$$

(as long as η is not too large)

Final Test Function

Upshot: Easy to sample from approximate leverage distribution for the Gaussian kernel with $x_1, \dots, x_n \in [-\delta, \delta]^d$:

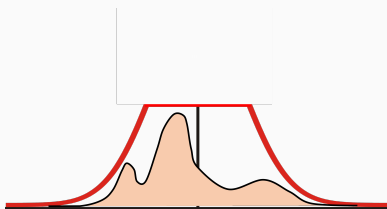
$$\bar{\tau}_\lambda(\boldsymbol{\eta}) = \begin{cases} \tilde{O}(\delta) & \text{when } \boldsymbol{\eta} \leq \sqrt{\log n / \lambda} \\ p(\boldsymbol{\eta}) = e^{-\|\boldsymbol{\eta}\|_2^2 / 2} & \text{otherwise.} \end{cases}$$



Final Test Function

Upshot: Easy to sample from approximate leverage distribution for the Gaussian kernel with $x_1, \dots, x_n \in [-\delta, \delta]^d$:

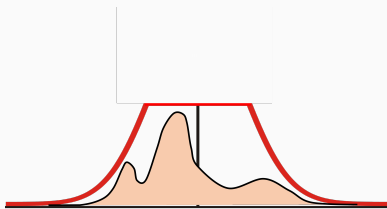
$$\bar{\tau}_\lambda(\boldsymbol{\eta}) = \begin{cases} \tilde{O}(\delta) & \text{when } \boldsymbol{\eta} \leq \sqrt{\log n / \lambda} \\ p(\boldsymbol{\eta}) = e^{-\|\boldsymbol{\eta}\|_2^2 / 2} & \text{otherwise.} \end{cases}$$



Final Test Function

Upshot: Easy to sample from approximate leverage distribution for the Gaussian kernel with $x_1, \dots, x_n \in [-\delta, \delta]^d$:

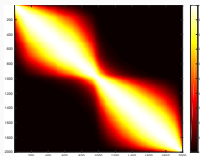
$$\bar{\tau}_\lambda(\boldsymbol{\eta}) = \begin{cases} \tilde{O}(\delta) & \text{when } \boldsymbol{\eta} \leq \sqrt{\log n / \lambda} \\ p(\boldsymbol{\eta}) = e^{-\|\boldsymbol{\eta}\|_2^2 / 2} & \text{otherwise.} \end{cases}$$



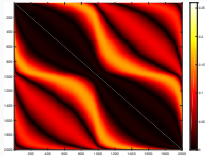
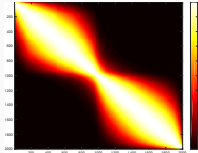
Requires $O(\delta \sqrt{\log(n/\lambda)}) \cdot \frac{1}{\epsilon^2}$ samples for spectral guarantee.

(vs. $O(n/\lambda) \cdot \frac{1}{\epsilon^2}$ for standard random Fourier features.)

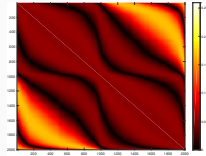
Gaussian kernel for two clusters:



Gaussian kernel for two clusters:

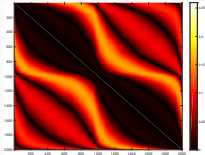
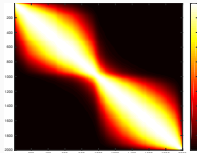


Standard RFF element error.

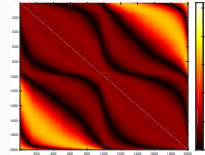


Modified RFF element error.

Gaussian kernel for two clusters:



Standard RFF element error.



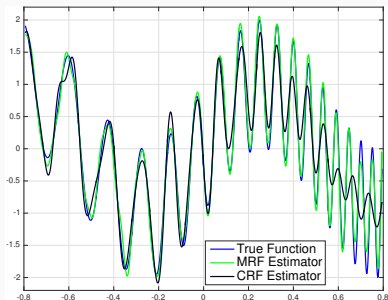
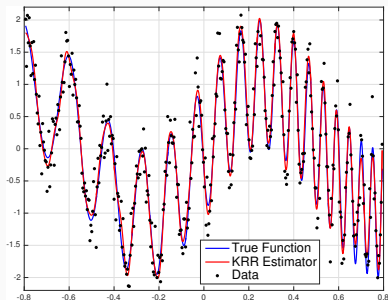
Modified RFF element error.

$$(1 - \epsilon)(\mathbf{Z}\mathbf{Z}^* + \lambda\mathbf{I}) \preceq \mathbf{K} + \lambda\mathbf{I} \preceq (1 + \epsilon)(\mathbf{Z}\mathbf{Z}^* + \lambda\mathbf{I}).$$

Sampling low frequencies relatively less biases error to align with large eigenvectors.

Experimental Results

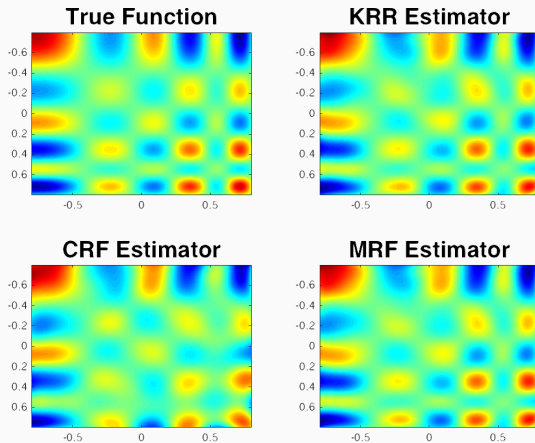
Example of approximate kernel ridge regression to interpolate a synthetic function:



CRF = classic random Fourier features ‘column norm’ sampling,
MRF = our modified sampling distribution.

Experimental Results

In higher dimensions:



CRF = classic random Fourier features 'column norm' sampling,
MRF = our modified sampling distribution.

Summary

Summary

- Viewed random Fourier features methods as a **matrix sampling problem**.

Summary

- Viewed random Fourier features methods as a **matrix sampling problem**.
- Used optimization perspective on ridge leverage scores to certify upper bounds on these scores for bounded datasets.
Reduced score computation to **Fourier approximation problem**.

Summary

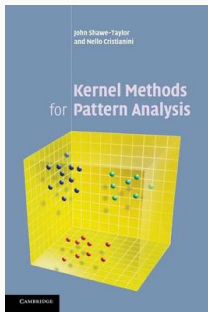
- Viewed random Fourier features methods as a **matrix sampling problem**.
- Used optimization perspective on ridge leverage scores to certify upper bounds on these scores for bounded datasets. Reduced score computation to **Fourier approximation problem**.
- New sampling distribution under-samples lower frequencies to obtain better low-rank approximations to the kernel matrix.

Algorithmic Challenge

Kernel methods are widely applied at all scales.

Algorithmic Challenge

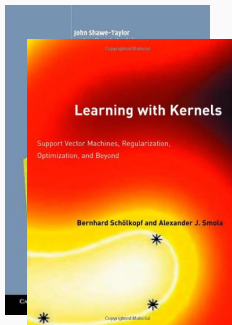
Kernel methods are widely applied at all scales.



data analysis
bread-and-butter

Algorithmic Challenge

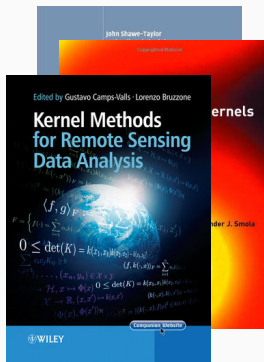
Kernel methods are widely applied at all scales.



data analysis
bread-and-butter

Algorithmic Challenge

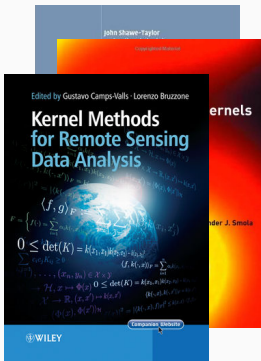
Kernel methods are widely applied at all scales.



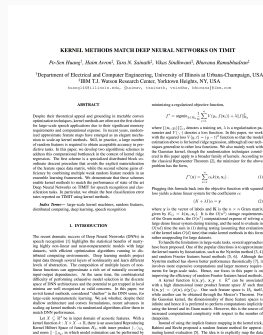
data analysis
bread-and-butter

Algorithmic Challenge

Kernel methods are widely applied at all scales.



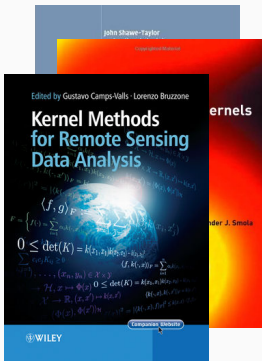
data analysis
bread-and-butter



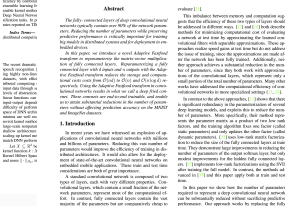
potential for new
state-of-the-art

Algorithmic Challenge

Kernel methods are widely applied at all scales.



data analysis
bread-and-butter



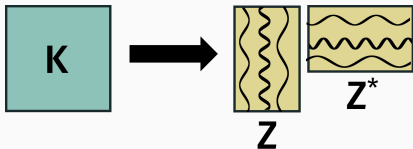
potential for new
state-of-the-art

Open Question

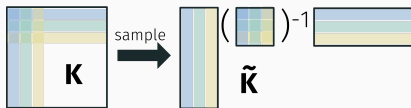
Open Question:

Kernel approximation methods you'll find in e.g., `scikit-learn`:

Random Feature Methods



Nyström Methods

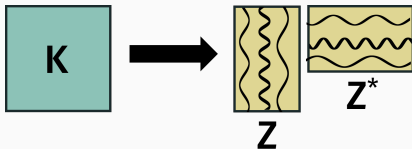


Open Question

Open Question:

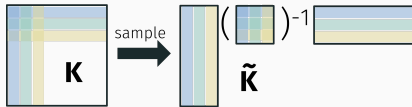
Kernel approximation methods you'll find in e.g., `scikit-learn`:

Random Feature Methods



- Good accuracy, less developed theory.
- Very fast.

Nystrom Methods



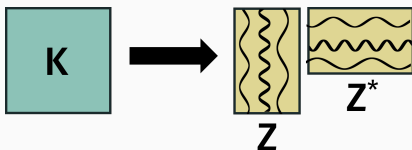
- Very good theoretical and empirical accuracy.
- Getting faster (see e.g. [M, Musco NIPS '17])

Open Question

Open Question:

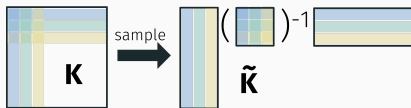
Kernel approximation methods you'll find in e.g., `scikit-learn`:

Random Feature Methods



- Good accuracy, less developed theory.
- Very fast.

Nystrom Methods



- Very good theoretical and empirical accuracy.
- Getting faster (see e.g. [M, Musco NIPS '17])

Can either class of methods obtain optimal speed and accuracy?

Thank you!